

# PS04 - OLS Sample Distribution

Zachary Tipton

2026-03-02

## OLS Sampling Distributions

In this problem set, we will use simulation to study the sampling distribution of OLS estimates. We will start with a baseline model and then change sample size, error variance, and the covariance of regressors.

Finally, we will study the coverage of confidence intervals under different error structures and standard error choices.

```
library(fixest)
library(ggplot2)
library(patchwork)
library(mvtnorm)

## Function to take a matrix of estimates and plot the marginal sample distributions and the
plot_ests <- function(est1, est2) {
  # Marginal sample distribution
  hist_est1 <- ggplot() +
    geom_histogram(aes(x = est1), bins = 50) +
    labs(x = "First estimate", y = NULL) +
    theme_minimal()

  hist_est2 <- ggplot() +
    geom_histogram(aes(x = est2), bins = 50) +
    labs(x = "Second estimate", y = NULL) +
    theme_minimal()

  # Joint sampling distribution of  $\hat{\beta}_1$  and  $\hat{\beta}_2$ 
  plot_joint <- ggplot() +
    geom_density_2d(aes(x = ests[, 2], y = ests[, 3]), bins = 50) +
```

```

labs(
  x = "First estimate",
  y = "Second estimate",
) +
scale_fill_viridis_c(guide = "none") +
theme_minimal()

((hist_est1 + hist_est2) / plot_joint) +
plot_layout(heights = c(1, 1.5))
}

```

## Baseline Simulation

We begin with a simple data-generating process where  $X_1$  and  $X_2$  are independent standard normals, the error is iid normal, and the true coefficients are (10, 1, 1).

The plots below show the marginal and joint sampling distributions of the OLS estimates.

For each simulation, we: 1. Generate data and estimate OLS with `feols`. 2. Store the estimated coefficients in a row of the matrix `ests` 3. Plot the marginal distribution of each estimate and then the joint distribution of the two estimates

```

# BASELINE CASE (for comparison)
B <- 1000
n <- 100
ests <- matrix(nrow = B, ncol = 3)
for (b in 1:B) {
  # Generate data
  Sigma_X = matrix(c(1, 0, 0, 1), nrow = 2, ncol = 2)
  X <- rmvnorm(n = n, mean = c(1, 1), sigma = Sigma_X)
  X1 <- X[, 1]
  X2 <- X[, 2]

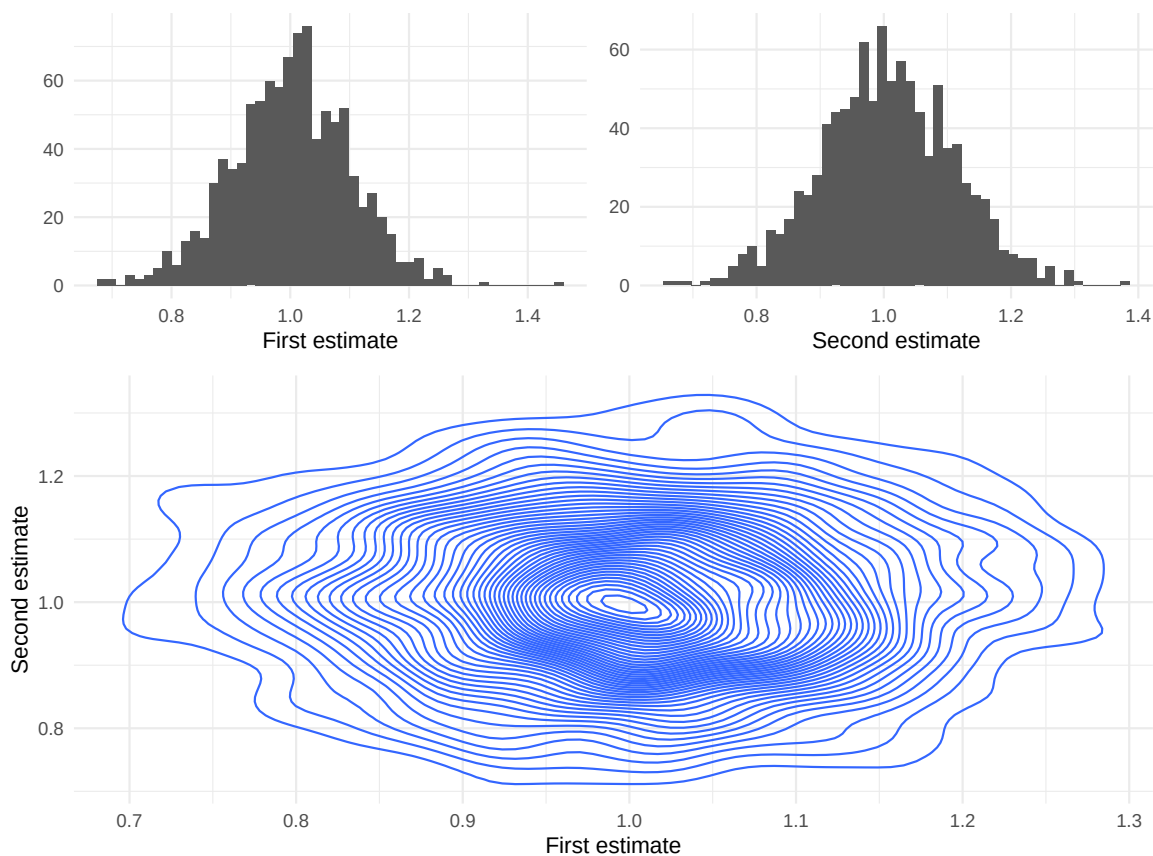
  y <- 10 + X1 * 1 + X2 * 1 + rnorm(n, mean = 0, sd = 1)

  df_b <- data.frame(X1 = X1, X2 = X2, y = y)

  # Estimate OLS
  mod <- feols(y ~ 1 + X1 + X2, data = df_b)
  ests[b, ] <- coef(mod)
}

```

```
plot_ests(ests[, 2], ests[, 3])
```



## 2. Changing sample size

Next, we change the sample size while keeping the data-generating process fixed. This shows how sampling variability shrinks as  $n$  grows.

```
B <- 1000
n <- 10 # here we lower n from 1000 to 10
ests <- matrix(nrow = B, ncol = 3)
for (b in 1:B) {
  # Generate data
  Sigma_X = matrix(c(1, 0, 0, 1), nrow = 2, ncol = 2)
  X <- rmvnorm(n = n, mean = c(1, 1), sigma = Sigma_X)
  X1 <- X[, 1]
```

```

X2 <- X[, 2]

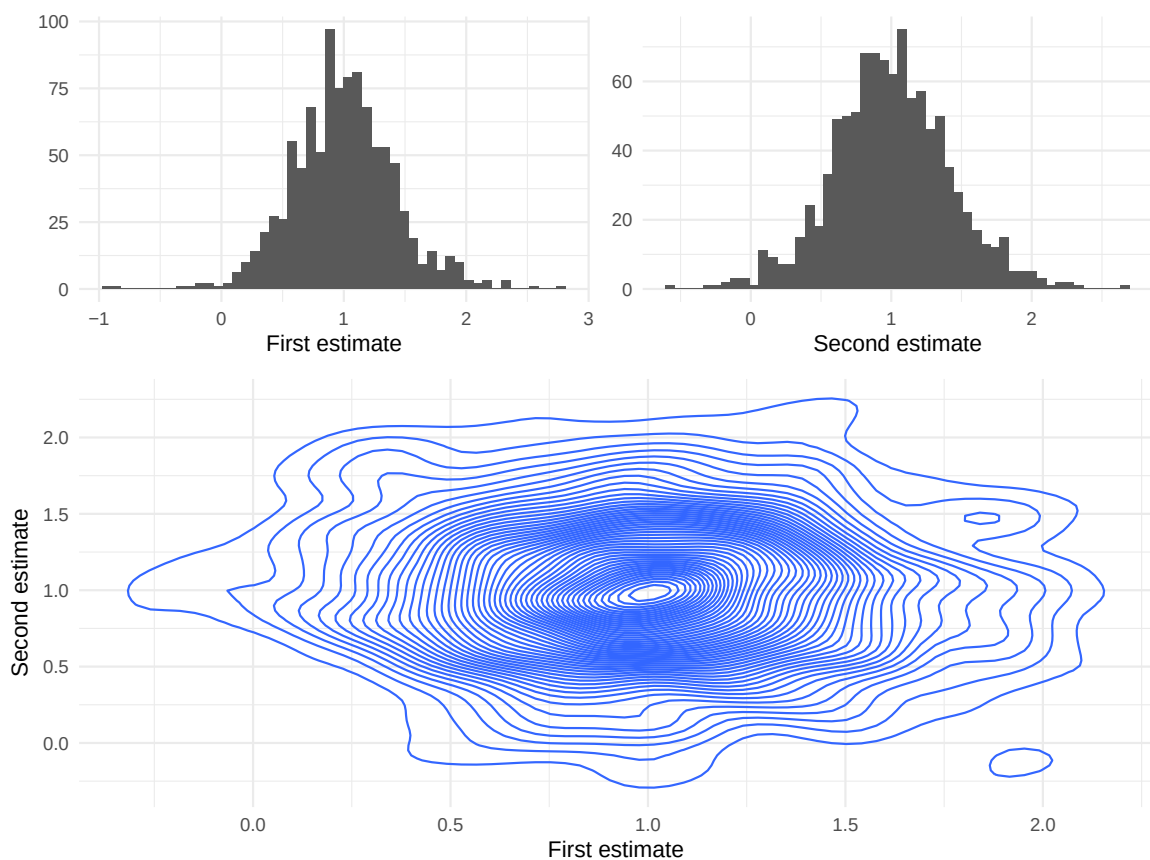
y <- 10 + X1 * 1 + X2 * 1 + rnorm(n, mean = 0, sd = 1)

df_b <- data.frame(X1 = X1, X2 = X2, y = y)

# Estimate OLS
mod <- feols(y ~ 1 + X1 + X2, data = df_b)
ests[b, ] <- coef(mod)
}

plot_ests(est[s[, 2], est[s[, 3]])

```



```

B <- 1000
n <- 1000 # back to 1000
ests <- matrix(nrow = B, ncol = 3)

```

```

for (b in 1:B) {
  # Generate data
  Sigma_X = matrix(c(1, 0, 0, 1), nrow = 2, ncol = 2)
  X <- rmvnorm(n = n, mean = c(1, 1), sigma = Sigma_X)
  X1 <- X[, 1]
  X2 <- X[, 2]

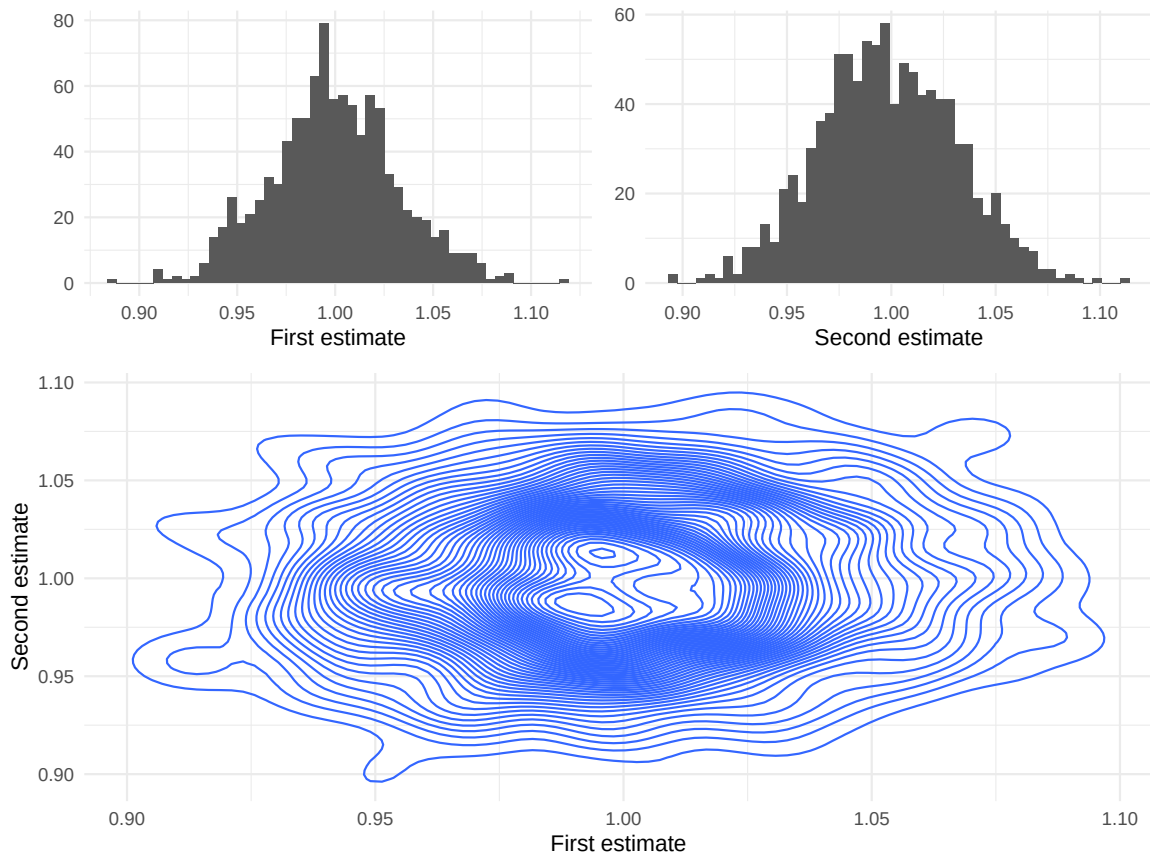
  y <- 10 + X1 * 1 + X2 * 1 + rnorm(n, mean = 0, sd = 1)

  df_b <- data.frame(X1 = X1, X2 = X2, y = y)

  # Estimate OLS
  mod <- feols(y ~ 1 + X1 + X2, data = df_b)
  ests[b, ] <- coef(mod)
}

plot_ests(ests[, 2], ests[, 3])

```



### Exercise

1. How does the sample size shape the sample distribution of each coefficient?

The tails get fatter as the model has little data to work with. The model will generate more values for the beta parameters deviating from the mean. The sampling distribution gets wider with increased variance.

2. Does it effect the shape of the joint distribution?

We see more “islands” show up in the joint distribution. This reflects the increased variation and more estimates falling into the tails of the distribution. The joint distribution also looks more “thin” at lower sample sizes.

### 3. Changing error term distribution

Now we will change the variance of the error term. This isolates how noise in  $y$  affects the precision of OLS estimates.

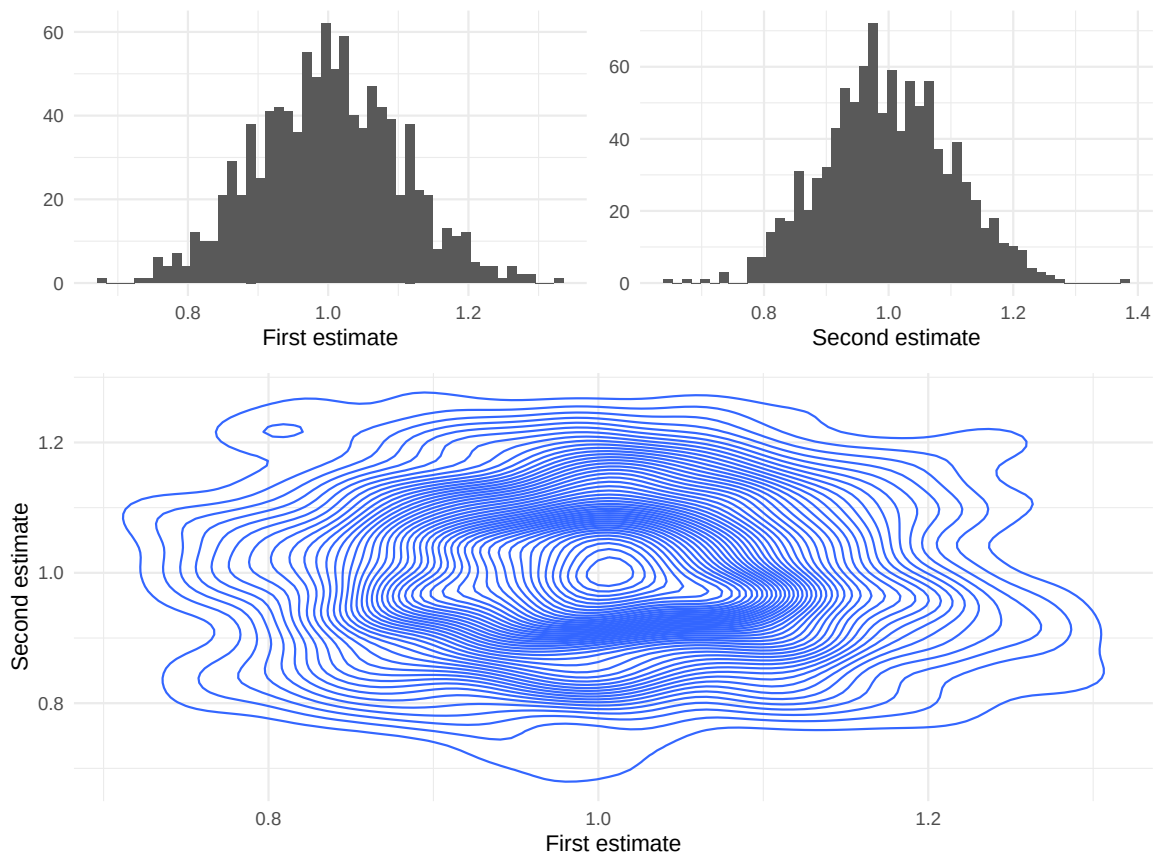
```
B <- 1000
n <- 100
ests <- matrix(nrow = B, ncol = 3)
for (b in 1:B) {
  # Generate data
  Sigma_X = matrix(c(1, 0, 0, 1), nrow = 2, ncol = 2)
  X <- rmvnorm(n = n, mean = c(1, 1), sigma = Sigma_X)
  X1 <- X[, 1]
  X2 <- X[, 2]

  y <- 10 + X1 * 1 + X2 * 1 + rnorm(n, mean = 0, sd = 1)

  df_b <- data.frame(X1 = X1, X2 = X2, y = y)

  # Estimate OLS
  mod <- feols(y ~ 1 + X1 + X2, data = df_b)
  ests[b, ] <- coef(mod)
}

plot_ests(ests[, 2], ests[, 3])
```



```

B <- 1000
n <- 100
ests <- matrix(nrow = B, ncol = 3)
for (b in 1:B) {
  # Generate data
  Sigma_X = matrix(c(1, 0, 0, 1), nrow = 2, ncol = 2)
  X <- rmvnorm(n = n, mean = c(1, 1), sigma = Sigma_X)
  X1 <- X[, 1]
  X2 <- X[, 2]

  y <- 10 + X1 * 1 + X2 * 1 + rnorm(n, mean = 0, sd = 3) # here we increase the error variance

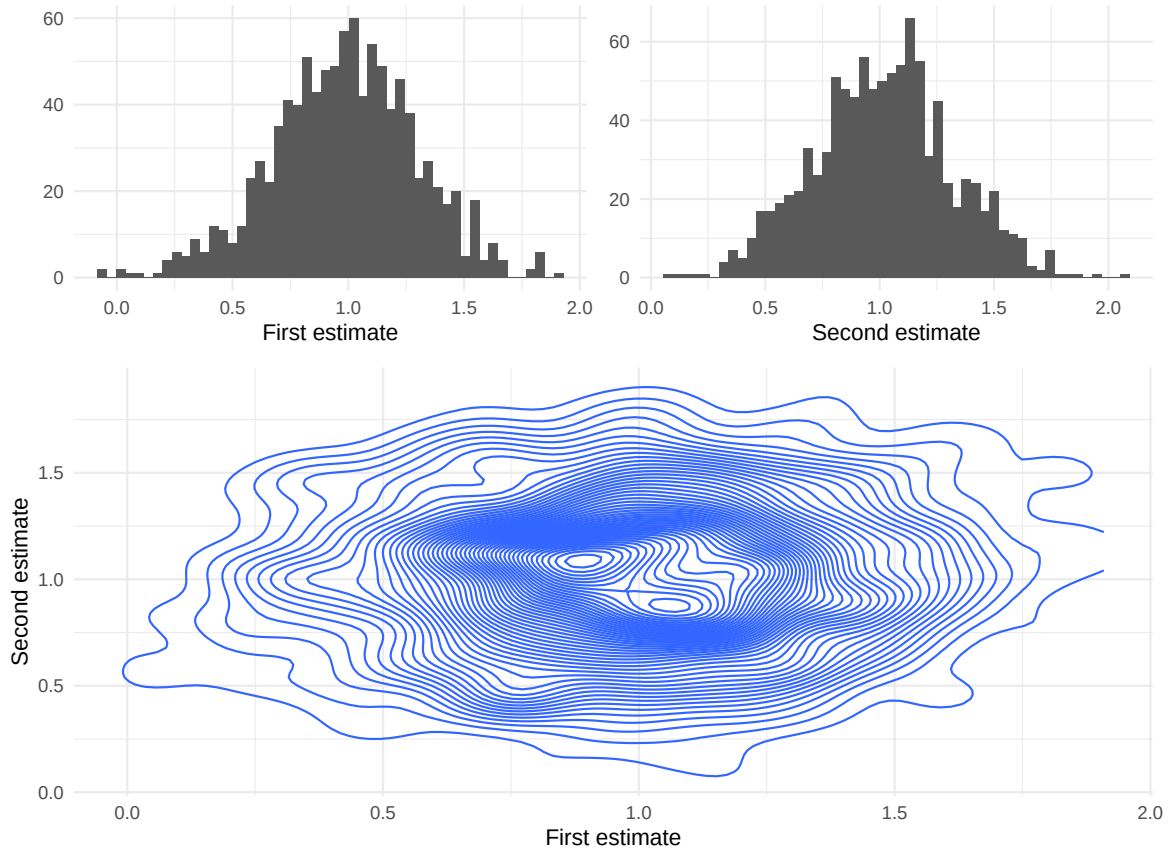
  df_b <- data.frame(X1 = X1, X2 = X2, y = y)

  # Estimate OLS
  mod <- feols(y ~ 1 + X1 + X2, data = df_b)
  ests[b, ] <- coef(mod)
}

```

```
}
```

```
plot_ests(ests[, 2], ests[, 3])
```



### Exercise

1. Comment on what happens to the standard deviation of the sample distribution and the joint distribution.

The histogram gets wider as the error variance increases. The joint distribution also gets wider. The estimates are more spread out and less precise.

## 4. Changing covariance matrix of $X$

Here we allow  $X_1$  and  $X_2$  to be correlated. We will do that by specifying non-zero off-diagonal elements of **Sigma\_X**, the variance-covariance matrix of the  $X$  variables. This illustrates how

multicollinearity changes the marginal and joint sampling distributions of the estimates.

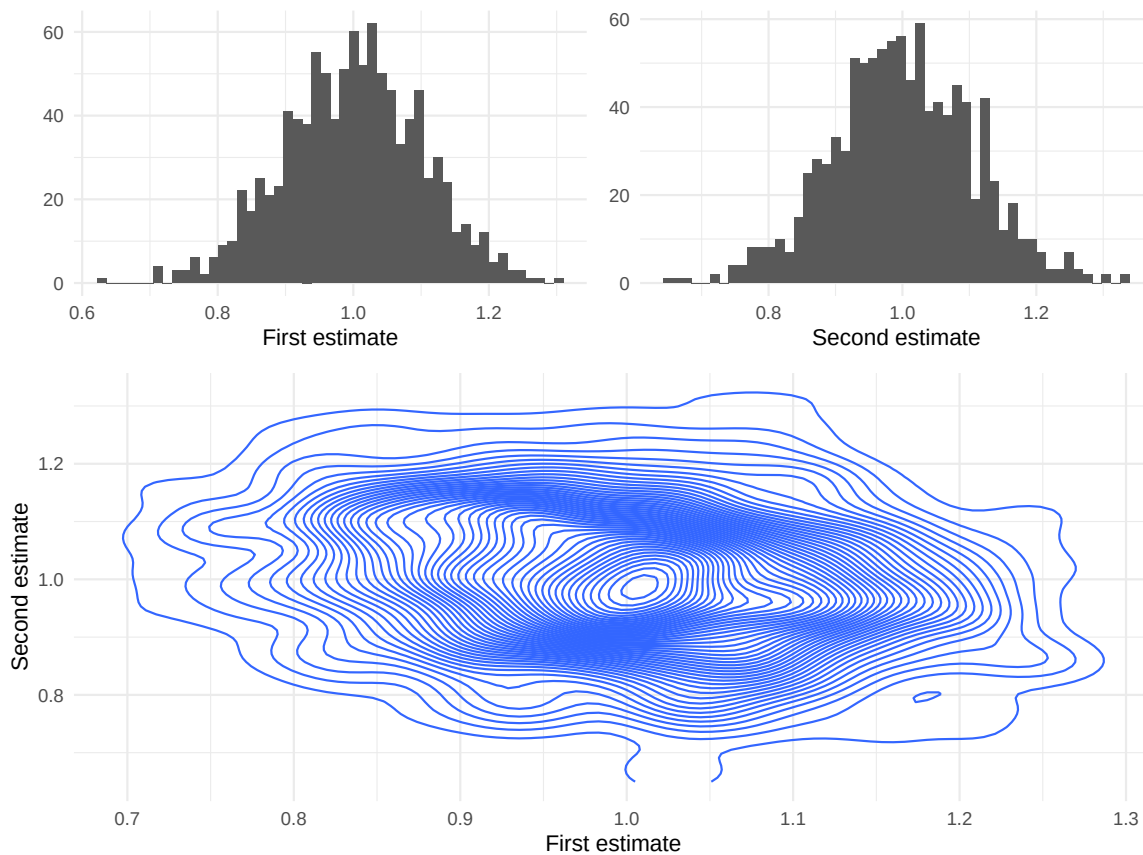
```
B <- 1000
n <- 100
ests <- matrix(nrow = B, ncol = 3)
for (b in 1:B) {
  # Generate data
  Sigma_X = matrix(c(1, 0.25, 0.25, 1), nrow = 2, ncol = 2) # here we set the covariance between X1 and X2
  X <- rmvnorm(n = n, mean = c(1, 1), sigma = Sigma_X)
  X1 <- X[, 1]
  X2 <- X[, 2]

  y <- 10 + X1 * 1 + X2 * 1 + rnorm(n, mean = 0, sd = 1)

  df_b <- data.frame(X1 = X1, X2 = X2, y = y)

  # Estimate OLS
  mod <- feols(y ~ 1 + X1 + X2, data = df_b)
  ests[b, ] <- coef(mod)
}

plot_ests(ests[, 2], ests[, 3])
```



```

B <- 1000
n <- 100
ests <- matrix(nrow = B, ncol = 3)
for (b in 1:B) {
  # Generate data
  Sigma_X = matrix(c(1, 0.99, 0.99, 1), nrow = 2, ncol = 2) # here we increase the covariance
  X <- rmvnorm(n = n, mean = c(1, 1), sigma = Sigma_X)
  X1 <- X[, 1]
  X2 <- X[, 2]

  y <- 10 + X1 * 1 + X2 * 1 + rnorm(n, mean = 0, sd = 1)

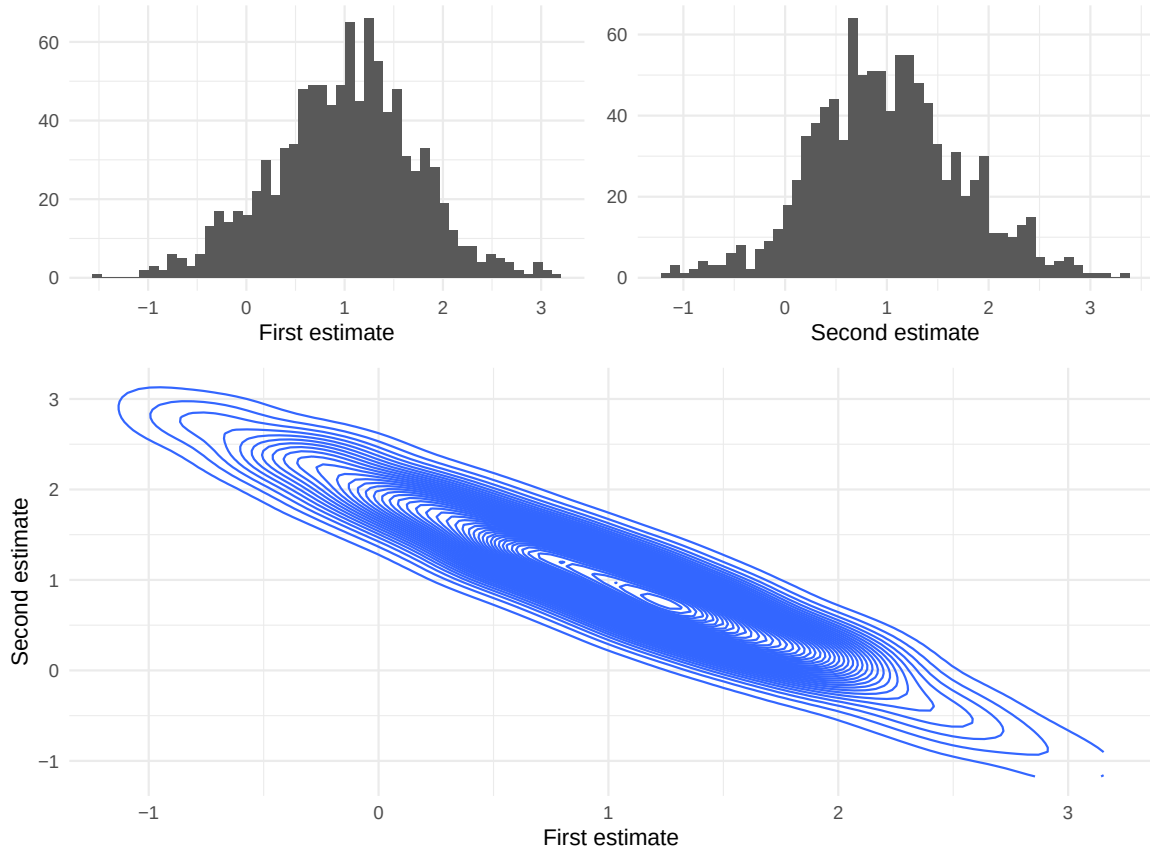
  df_b <- data.frame(X1 = X1, X2 = X2, y = y)

  # Estimate OLS
  mod <- feols(y ~ 1 + X1 + X2, data = df_b)
  ests[b, ] <- coef(mod)
}

```

```
}
```

```
plot_ests(ests[, 2], ests[, 3])
```



## Exercises

1. Comment on what happens to the standard deviation of the sample distribution and the joint distribution.

The standard deviation of the sample distribution increases as the covariance between  $X_1$  and  $X_2$  increases. The joint distribution tilts and becomes more elongated along the downward sloping line. This shows that they are highly negatively correlated.

2. For the case of an extreme positive correlation (0.99), I want you to think about why the results look the way they do. Try to explain why in your own words. Does this change as we increase  $n$ ?

As one variable increases, the other variable also increases. Basically, they are both tracking each other and the model doesn't know how to distinguish their individual effects.

Adding more data doesn't help, but will just cluster the estimates more tightly around the mean. The estimates will still be highly correlated and the joint distribution will still be elongated along the downward sloping line.

## 6. Forecasts with collinearity

Even when  $X_1$  and  $X_2$  are highly correlated, the linear combination that drives predictions can still be estimated precisely. In this example we use a nearly collinear design, estimate OLS repeatedly, and plot the distribution of forecasts at a fixed point.

```
B <- 5000
n <- 100
preds <- numeric(B)

for (b in 1:B) {
  Sigma_X = matrix(c(1, 0.99, 0.99, 1), nrow = 2, ncol = 2)
  X <- rmvnorm(n = n, mean = c(1, 1), sigma = Sigma_X)
  X1 <- X[, 1]
  X2 <- X[, 2]

  y <- 10 + X1 * 1 + X2 * 1 + rnorm(n, mean = 0, sd = 1)

  df_b <- data.frame(X1 = X1, X2 = X2, y = y)
  mod <- feols(y ~ 1 + X1 + X2, data = df_b)

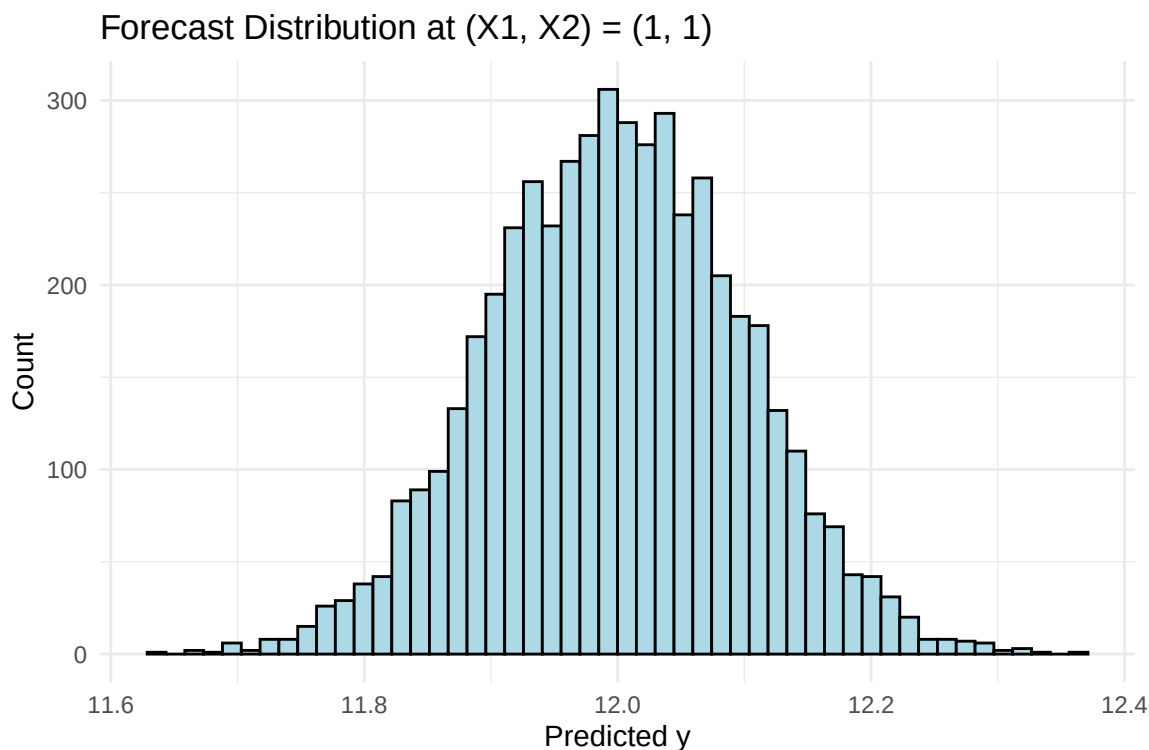
  x_new <- data.frame(X1 = 1, X2 = 1)
  preds[b] <- predict(mod, newdata = x_new)
}

ggplot() +
  geom_histogram(
    aes(x = preds),
    bins = 50,
    fill = "lightblue",
    color = "black"
  ) +
  labs(
    title = "Forecast Distribution at (X1, X2) = (1, 1)",
    x = "Predicted y",
```

```

y = "Count"
) +
theme_minimal()

```



Note that in this extreme case, the OLS coefficients seem to fall on the line  $\hat{\beta}_1 + \hat{\beta}_2 = 2$ . In the case of the  $Cov(X_1, X_2) = 0.99$ , the two are nearly collinear. Therefore, OLS is not sure if to give “credit” to  $X_2$  or  $X_1$  and OLS distributes their combined effect arbitrarily between the two betas. But, it is able to tell that a one unit increase in  $X_1$  (and therefore  $\approx$  a 1 unit increase in  $X_2$ ) yields a 2 unit increase in  $y$ . This means that when  $\hat{\beta}_1$  is large  $\hat{\beta}_2$  must be small or negative to compensate and make  $\hat{\beta}_1 + \hat{\beta}_2 = 2$ .

Even though the individual coefficient estimates are noisy under collinearity, the forecast is based on their sum. Because  $X_1$  and  $X_2$  move together, the combined effect is more stable than each coefficient by itself.

## Coverage of Confidence Intervals

When running regressions, it is important to think about how to estimate your standard errors. In class, we discuss that the difficult part is estimating the variance-covariance matrix of

individuals' error terms:  $\sigma_{i,j} \equiv \text{Cov}(\varepsilon_i, \varepsilon_j)$ . The three most common are: - homoskedastic ( $\sigma_{i,i} = \sigma^2$ ) and independent ( $\sigma_{i,j} = 0$ ) - heteroskedastic ( $\sigma_{i,i} = \sigma_i^2$ ) and independent ( $\sigma_{i,j} = 0$ ) - clustered ( $\sigma_{i,j} \neq 0$  if  $i$  and  $j$  are in the same cluster) and  $\sigma_{i,j} = 0$  otherwise.

We will check how often a confidence interval covers the true parameter value. The data-generating process can have iid, heteroskedastic, or clustered errors. The standard error type used in the confidence interval is controlled separately.

The function below takes two key arguments: - **error\_type** controls the true error process in the DGP. - **se\_type** controls how standard errors are estimated for the confidence interval.

For each simulation, we: 1. Generate data and estimate OLS with **feols**. 2. Construct a confidence interval using the requested standard errors. 3. Record whether the true coefficient is inside that interval. (coverage) 4. Return the empirical coverage rate.

```
coverage_simulation <- function(
  B = 2000,
  n = 100,
  beta_0 = 10,
  beta_1 = 1,
  beta_2 = 1,
  error_type = c("iid", "heteroskedastic", "cluster"),
  se_type = "iid",
  alpha = 0.05
) {
  error_type <- match.arg(error_type)
  if (n %% 20 != 0) {
    stop("`n` needs to be divisible by 20")
  }
  g <- rep(1:(n / 20), each = 20) ## classes of size 20

  cover <- logical(B)
  for (b in 1:B) {
    Sigma_X = matrix(c(1, 0, 0, 1), nrow = 2, ncol = 2)
    X <- rmvnorm(n = n, mean = c(1, 1), sigma = Sigma_X)
    X1 <- X[, 1]
    X2 <- X[, 2]

    if (error_type == "heteroskedastic") {
      eps_sd <- 1 + 1.5 * abs(X1)
      eps <- rnorm(n, mean = 0, sd = eps_sd)
    } else {
      eps <- rnorm(n, mean = 0, sd = 1)
    }
  }
}
```

```

## Generate common shocks for each class
fe_g <- if (error_type == "cluster") {
  class_shock <- rnorm(max(g), mean = 0, sd = 1)
  class_shock[g]
} else {
  0
}

y <- beta_0 + beta_1 * X1 + beta_2 * X2 + fe_g + eps
df_b <- data.frame(X1 = X1, X2 = X2, y = y, g = g)

mod <- feols(y ~ 1 + X1 + X2, data = df_b, notes = FALSE)

vcov_mat <- vcov(mod, vcov = se_type)
beta_hat <- coef(mod)["X1"]
se_hat <- sqrt(vcov_mat["X1", "X1"])
# z <- qnorm(1 - (alpha / 2))
z <- qt(1 - (alpha / 2), df = degrees_freedom(mod, type = "resid")) ## More precise
ci <- c(beta_hat - z * se_hat, beta_hat + z * se_hat)
cover[b] <- (beta_1 >= ci[1]) && (beta_1 <= ci[2])
}

mean(cover)
}

```

```

## Examples
set.seed(20260224)
coverage_simulation(n = 100, error_type = "iid", se_type = "iid")

```

```
[1] 0.9545
```

```
coverage_simulation(n = 100, error_type = "heteroskedastic", se_type = "iid")
```

```
[1] 0.8785
```

```
coverage_simulation(n = 100, error_type = "heteroskedastic", se_type = "hc1")
```

```
[1] 0.9375
```

```
coverage_simulation(n = 100, error_type = "cluster", se_type = ~g) # 5 groups of 20
```

```
[1] 0.8615
```

```
coverage_simulation(n = 500, error_type = "cluster", se_type = ~g) # 25 groups of 20
```

```
[1] 0.934
```

```
coverage_simulation(n = 1000, error_type = "cluster", se_type = ~g) # 50 groups of 20
```

```
[1] 0.949
```

### Exercises

1. In the coverage simulation, keep `error_type = "iid"` and compare `se_type = "iid"`, `se_type = "hc1"`, and `se_type = ~g` (clustered). Do the coverage rates differ? Why or why not?

```
coverage_simulation(n = 100, error_type = "iid", se_type = "iid")
```

```
[1] 0.9555
```

```
coverage_simulation(n = 100, error_type = "iid", se_type = "hc1")
```

```
[1] 0.9275
```

```
coverage_simulation(n = 100, error_type = "iid", se_type = ~g)
```

```
[1] 0.896
```

`error_type = "iid"` and `se_type = "iid"` should give us good coverage (here 94%) because the standard error estimation matches the true error structure.

`error_type = "iid"` and `se_type = "hc1"` should similarly perform well, because the more robust standard error estimator is still consistent under homoskedasticity. Our result is 94% coverage.

`error_type = "iid"` and `se_type = ~g` performs at 88% coverage, which is much worse than the other two. This is likely because the clustered standard error estimator is trying to estimate clustered error structure when there is none.

2. Set `error_type = "heteroskedastic"` and compare `se_type = "iid"` to `se_type = "hc1"`. Which one is closer to 95% coverage? Explain in a sentence.

```
coverage_simulation(n = 100, error_type = "heteroskedastic", se_type = "iid")
```

```
[1] 0.8785
```

```
coverage_simulation(n = 100, error_type = "heteroskedastic", se_type = "hc1")
```

```
[1] 0.935
```

`se_type = "hc1"` is closer to 95% coverage (94%) than `se_type = "iid"` (86%) because the “hc1” standard error estimator is robust to heteroskedasticity, while the “iid” standard error estimator is not.

3. Increase the sample size in exercise 2. Does this fix the problem? Explain why or why not.

```
coverage_simulation(n = 1000, error_type = "heteroskedastic", se_type = "iid")
```

```
[1] 0.8865
```

```
coverage_simulation(n = 1000, error_type = "heteroskedastic", se_type = "hc1")
```

```
[1] 0.944
```

Increasing the sample size slightly improves the coverage of `se_type = "iid"` (from 86% to 88%), but it is still the incorrect standard error estimator. The coverage of `se_type = "hc1"` remains good (94%).

4. Set `error_type = "cluster"` and compare `se_type = "iid"` to `se_type = ~g`. Which one is closer to 95% coverage? What is the role of clustering here?

```
coverage_simulation(n = 100, error_type = "cluster", se_type = "iid")
```

```
[1] 0.9445
```

```
coverage_simulation(n = 100, error_type = "cluster", se_type = ~g)
```

```
[1] 0.865
```

At  $n = 100$ , `se_type = "iid"` is still performing well with almost 96% coverage, while `se_type = ~g` is performing worse with 88% coverage. However, increasing  $n$  to 1000 increased the coverage of `se_type = ~g` to 94%. Clustering breaks the assumption of independence of errors with a “class\_shock” that affects all observations in a cluster.

5. Increase the number of clusters by setting `n = 600` (30 groups of 20) in `coverage_simulation`. With `error_type = "cluster"` and `se_type = ~g`, how does the coverage rate change relative to `n = 100`?

```
coverage_simulation(n = 600, error_type = "cluster", se_type = ~g)
```

```
[1] 0.9415
```

I played around with this idea above by setting  $n$  to 1000, but didn’t know exactly how the cluster size was being determined. As we saw, increasing the number of clusters to 30 improves the coverage of `se_type = ~g` to 94%. This is because clustered standard errors require a sufficient number of clusters to perform well.

6. Bonus: small-sample inference. We know that when  $n$  is small, our standard normal approximation performs relatively poorly. Even though our error is generated from the normal distribution, the HC1 standard error tends to be slightly too small (though it converges as  $n$  grows). A solution is to use “HC3” standard errors (careful: this can be quite slow if  $n$  is large, but then again you don’t need this when  $n$  is large). E.g.:

```
coverage_simulation(n = 20, error_type = "heteroskedastic", se_type = "hc1")
```

```
[1] 0.891
```

```
coverage_simulation(n = 20, error_type = "heteroskedastic", se_type = "hc3")
```

```
[1] 0.94
```

## Key Takeaways

- Sampling distributions of OLS estimates tighten as sample size grows, but they widen with more noise in the error term.
- Correlation among regressors creates collinearity, which inflates variance and induces correlation across coefficient estimates.
- Confidence interval coverage depends on whether standard errors match the true error structure; robust or clustered SEs improve coverage when assumptions fail.
- Clustered standard errors require the number of groups to grow ( $G > 30$  instead of  $N > 30$ ). If you have few groups, can look into (wild) cluster bootstrap.